Compute closer to the user

Make exploration immediate. Use the backend to validate, authorize and record the final business decision.

BROWSER: explore locally

Move sliders, compare scenarios, filter loaded data and refresh charts without a request for every interaction.

BACKEND: establish authority

Recalculate submitted prices, enforce permissions and approval policies, persist records and maintain audit history.

Illustrative request reduction

Same journey: load pricing inputs, make 30 adjustments, then submit once. Only the location of preview calculations changes.

30 fewer calls per journey

93.75% fewer requests in this example: 32 down to 2.

Counting: server-preview design = 1 input fetch + 30 previews + 1 submission. Browser-preview design = 1 input fetch + 1 submission. Both designs still validate the submission on the server.

Illustration, not a benchmark: excludes authentication, refreshes, autosave and other traffic. Fewer calls do not imply an equal percentage reduction in total backend cost.

A fast preview. A trusted result.

Fetch approved inputs once, explore repeatedly in the browser, and validate the chosen scenario centrally.

BROWSER / React UI

1. Load permitted pricing inputs and rule version.

2. Explore fees, margins and scenario comparisons locally.

3. Submit chosen inputs and version.

API →

BACKEND / Java API

Supply authorized, versioned input data.

Recalculate the authoritative result; check access and discount limits.

Persist the scenario and approval state.

Key boundary: browser results are previews. The server never accepts a client-calculated price as proof of correctness.

Best opportunities to reduce backend chatter

Scenario sliders	Recalculate each adjustment locally. Save or submit the selected scenario.
Compare and chart	Transform already loaded scenario data. Fetch fresh data within user permissions.
Grid exploration	Sort, filter and group loaded rows. Search and paginate large datasets.
Sensitivity analysis	Evaluate permitted input combinations. Validate the chosen business outcome.

Data boundary: only send inputs the user may inspect. Hidden frontend code cannot protect confidential internal cost models. Keep sensitive calculations server-side.

Make the benefit measurable

Start with ordinary browser computation. Add heavier technology when profiling demonstrates the need.

JavaScript

Simple fee previews, validation, chart transforms and loaded-data filtering. Use decimal-safe money handling.

Web Workers

Move substantial calculations off the UI thread to preserve smooth interaction. They do not inherently reduce API calls.

WASM / WebGPU

Consider for suitable CPU-heavy or parallel workloads. Typical fee formulas rarely justify this complexity.

Production safeguards

Correct money: align decimal precision, currency and rounding with the backend.
Versioned inputs: submit the rule version; detect stale inputs and require a refreshed preview when needed.
Consistent results: run shared test vectors across browser and server calculations.
Safe execution: enforce authorization and approval limits server-side; support lower-powered devices.
Controlled traffic: debounce remote search, batch requests and cache with explicit freshness rules.

Measure before and after

Track requests per completed journey, transferred bytes, preview latency (p50/p95), UI responsiveness and browser/server result mismatches. Measure server CPU and cost separately.

ARCHITECTURE PRINCIPLE

Move repeated scenario exploration into the browser using permitted, versioned inputs. The backend independently recalculates the selected scenario and enforces approval controls. Measure request reduction, responsiveness and calculation consistency before expanding the approach.